

UNIVERSIDAD SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
CARRERA DE CIENCIAS Y SISTEMAS
SEGUNDO SEMESTRE 2026
ORGANIZACIÓN DE LENGUAJES Y
COMPILADORES 2
ING. MARIO RAMIREZ



MANUAL DE USUARIO PROYECTO FASE 1
(Nexus Solairy)

Erikson Denilson Orozco Monterroso 202131770

Quetzaltenango, 22 de Septiembre de 2026

INTRODUCCIÓN.....	3
1. CARACTERÍSTICAS PRINCIPALES.....	3
2. REQUISITOS DEL SISTEMA.....	3
Mínimos.....	3
Recomendados.....	4
INSTALACIÓN.....	4
Opción A: Ejecución mediante archivo JAR.....	4
Opción B: Ejecución mediante Imagen Nativa (Recomendada).....	4
Generación del Ejecutable.....	5
3. PRIMERA EJECUCIÓN Y VISTAS DEL IDE.....	6
3.1. Pantalla de Bienvenida y Gestión de Proyectos.....	6
3.2. Estructura de Directorios del Proyecto.....	8
3.3. Barra de Menú Principal y Opciones del IDE.....	8
3.4. Barra de Herramientas de Acceso Rápido.....	11
3.5. Editor de Código Multilenguaje.....	12
3.6. Panel Inferior de Herramientas y Depuración.....	13
Panel de Problemas y Diagnósticos.....	14
Visualizador del Árbol Sintáctico (AST).....	15
Inspector del Analizador Léxico.....	16
Visor de Código de Tres Direcciones (C3D).....	17
Barra de Estado Inferior.....	19

INTRODUCCIÓN

El presente manual de usuario constituye la guía técnica oficial para el uso e implementación del entorno de desarrollo e interpretación de datos estelares Nexus Solairy. Este documento detalla las especificaciones del sistema, procedimientos de instalación, estructura de archivos y funcionalidades avanzadas de análisis léxico, sintáctico y semántico.

Nexus Solairy es un entorno de desarrollo integrado (IDE) diseñado para el procesamiento y traducción de lenguajes de programación y protocolos estelares. Soporta la edición y análisis sintáctico de código en dialectos Python (.y), Zetariano (.z) y PigLatin (.pig). Además, integra módulos para la generación de Código de Tres Direcciones (C3D), construcción y renderizado del Árbol de Sintaxis Abstracta (AST), inspección de memoria (Stack y Heap) y detección de errores en tiempo de compilación.

1. CARACTERÍSTICAS PRINCIPALES

El sistema ofrece las siguientes capacidades técnicas esenciales:

- Soporte multilenguaje: Análisis y resaltado sintáctico para Python (.y), Zetariano (.z) y PigLatin (.pig).
- Generación e inspección de Código de Tres Direcciones (C3D) y estructuras de cuádruplas.
- Generación de grafos interactivos del Árbol de Sintaxis Abstracta (AST) integrados mediante Graphviz.
- Interfaz de usuario extensible desarrollada sobre JavaFX con tematización por dialecto.
- Módulos de diagnóstico para el análisis de errores léxicos, sintácticos y semánticos.

2. REQUISITOS DEL SISTEMA

Para garantizar un rendimiento óptimo de *Nexus Solairy*, el sistema debe cumplir con las siguientes especificaciones:

Mínimos

- **Sistema Operativo:** Windows, macOS o Linux (distribución compatible).
- **Memoria RAM:** Mínimo de 4 GB disponibles.

- **Entorno de Ejecución:** Java 21 o superior (si no se desea usar la imagen ver la instalación).
- **Espacio en disco:** 1 GB SSD
- **Herramientas Externas:** Graphviz instalado y configurado en el sistema para la generación del AST.

Recomendados

- **Memoria RAM:** 8 GB
- **Tarjeta Gráfica:** Aceleración por hardware para renderizado gráfico.
- **Herramientas de construcción:** Maven 3.8+ o superior.

INSTALACIÓN

Nexus Solairy proporciona dos métodos de despliegue según el entorno operativo de destino:

Opción A: Ejecución mediante archivo JAR

1. Accede al repositorio del Proyecto 1: <https://github.com/EDK367/nexus-solairy>
2. Descarga el artefacto binario (.jar) desde la sección de "Releases".

Ejecute el compilador desde la terminal utilizando la línea de comandos:

Shell

```
java -jar nexus-solairy-1.0-SNAPSHOT.jar
```

Opción B: Ejecución mediante Imagen Nativa (Recomendada)

1. Descarga el repositorio completo desde GitHub.
<https://github.com/EDK367/nexus-solairy>
2. Navega hacia el directorio principal de la aplicación e invoque el proceso de empaquetado.

Generación del Ejecutable

Para reconstruir la imagen binaria ejecutable directamente desde las fuentes del proyecto, ejecute las siguientes instrucciones en la terminal de comandos:

Shell

```
rm -rf dist
```

```
./mvnw clean package jpackage --type app-image --name  
NexusSolairy --app-version 1.0 --input target --main-jar  
nexus-solairy-1.0-SNAPSHOT.jar --main-class  
com.example.nexussolairy.Launcher --dest dist
```

Ejecución del aplicativo binario: Una vez completada la generación del paquete, ejecute la instrucción correspondiente al sistema operativo anfitrión:

Linux:

Shell

```
./dist/NexusSolairy/bin/NexusSolairy
```

Windows:

Shell

```
dist\NexusSolairy\NexusSolairy.exe
```

macOS:

Shell

```
open dist/NexusSolairy.app
```

Nota de compatibilidad: Se recomienda el uso de la imagen nativa generada mediante *jpackage* para maximizar el rendimiento y evitar incongruencias en las dependencias de *JavaFX*.

3. PRIMERA EJECUCIÓN Y VISTAS DEL IDE

Al iniciar la aplicación mediante el archivo ejecutable o la línea de comandos, se inicializa el entorno virtual de Nexus Solairy.

```
11m32s © java -jar nexus-solairy-1.0-SNAPSHOT.jar
o/Desktop java U v21.0.10 1
2:36
Sep 25, 2026 12:37:29 PM com.sun.javafx.application.PlatformImpl startup
WARNING: Unsupported JavaFX configuration: classes were loaded from 'unnamed module @19e01d37'
```

Figura 1: Consola de inicialización del entorno runtime.

Propósito: Mostrar el proceso de carga de librerías y la inicialización de la JVM y módulos JavaFX.

Elementos visuales: Ventana de consola en modo texto con logs de arranque y mensajes de advertencia de la plataforma.

Interacción: Vista pasiva e informativa de inicio del sistema.

3.1. Pantalla de Bienvenida y Gestión de Proyectos

Al completar la carga inicial, el usuario visualiza el diálogo flotante de bienvenida para la gestión de espacios de trabajo.

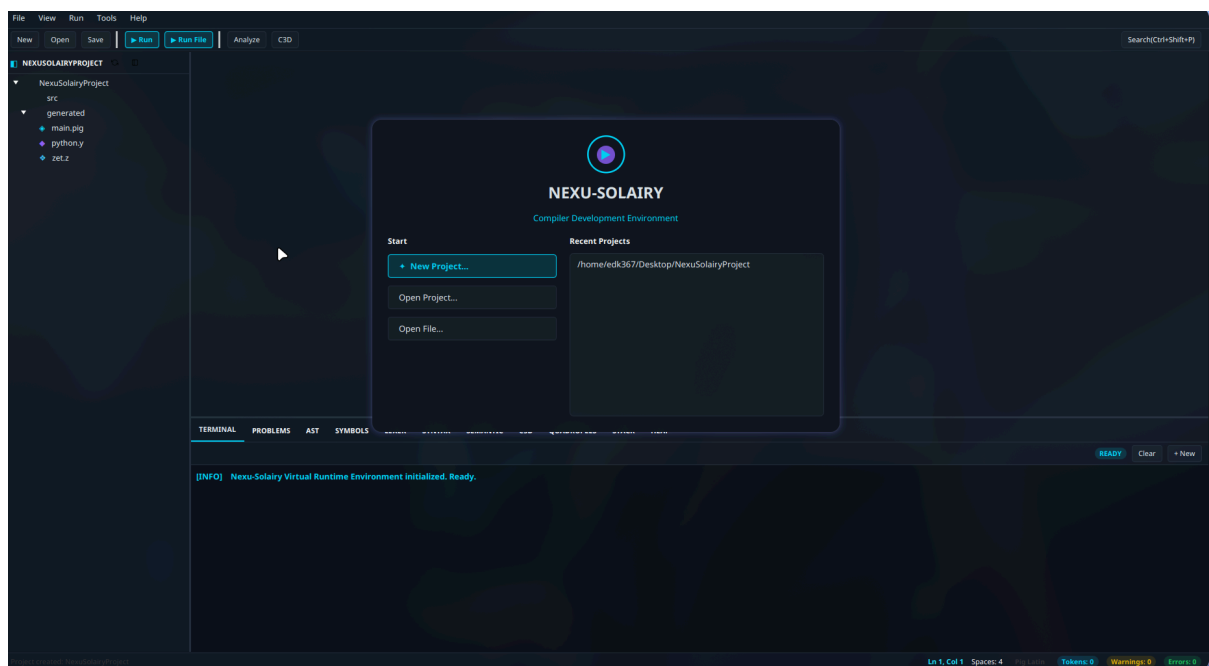


Figura 2: Diálogo flotante de bienvenida y selector de proyectos recientes.

Propósito: Permitir la creación rápida o navegación de proyectos de compilación existentes.

Elementos visuales: Panel central con botones de acción ("New Project...", "Open Project...", "Open File...") y lista lateral de proyectos recientes.

Interacción: Haga clic en "New Project..." para definir una nueva estructura, o seleccione un proyecto reciente de la lista.

Propósito: Permitir la creación rápida o navegación de proyectos de compilación.

Interacción: El usuario puede hacer clic en "New Project..." para definir una nueva estructura, o seleccionar un ítem del listado de la derecha para cargar proyectos previos.

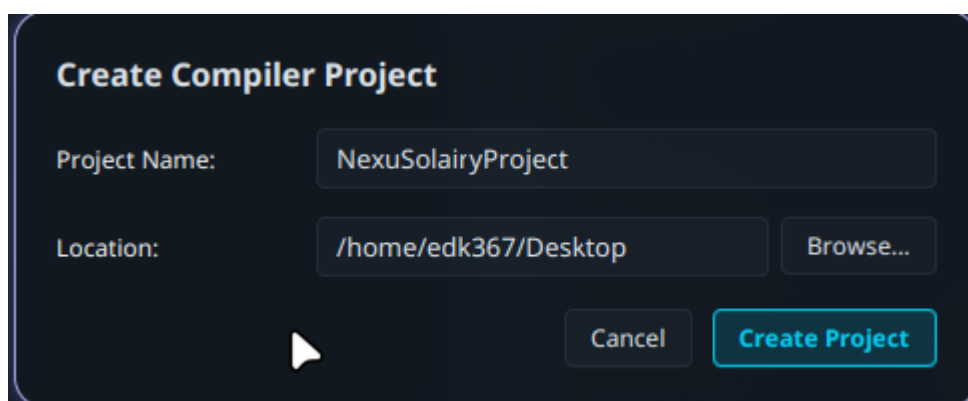
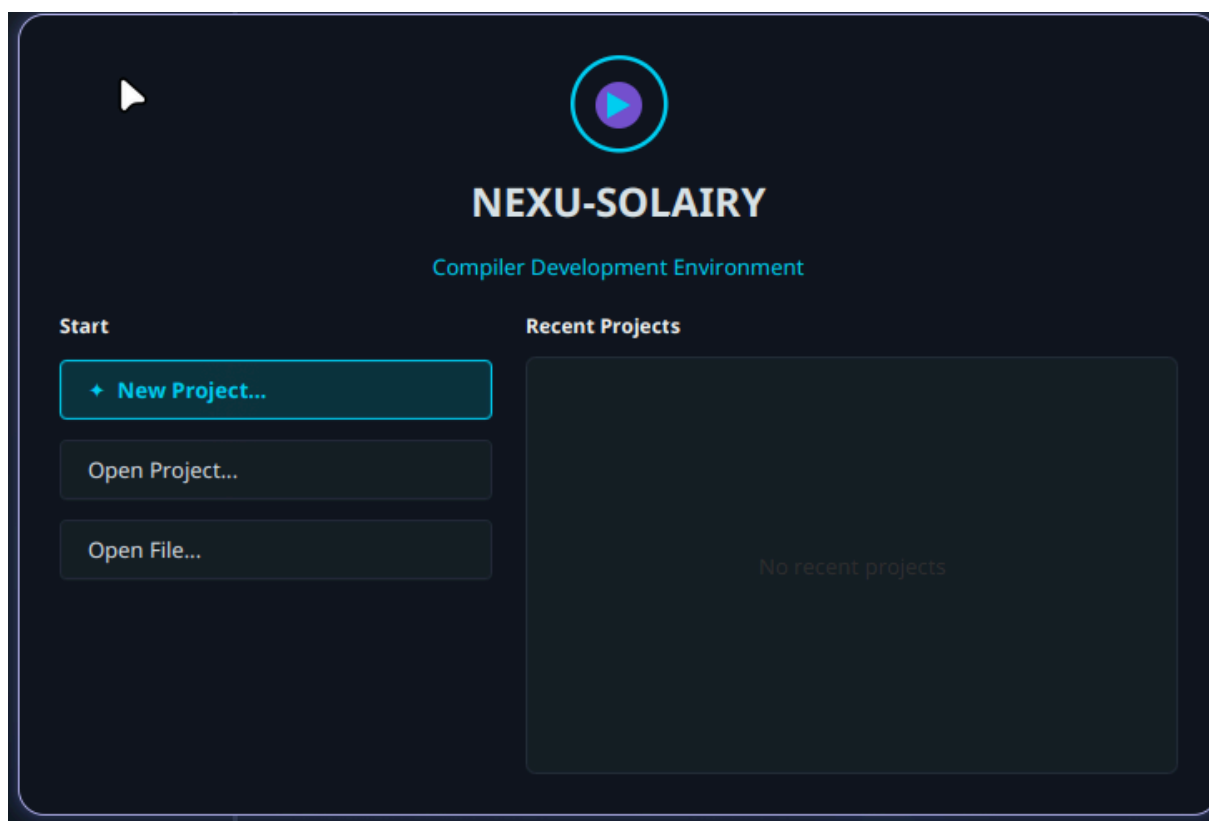


Figura 3: Asistente para la creación de proyectos ("Create Compiler Project").

Propósito: Configurar el identificador y la ruta de destino dentro del sistema de archivos.

Elementos visuales: Formulario modal con campos para "Project Name" y "Location", acompañado del botón "Browse...".

Interacción: Ingrese el nombre del proyecto, especifique la ruta mediante "Browse..." y haga clic en "Create Project" para confirmar.

3.2. Estructura de Directorios del Proyecto

Al generar un nuevo proyecto, el entorno automatiza la creación de los subdirectorios principales:

- **src/:** Aloja los archivos de código fuente primarios.

generated/: Contiene plantillas y archivos de prueba (main.pig, python.y, zet.z) para validación del compilador.

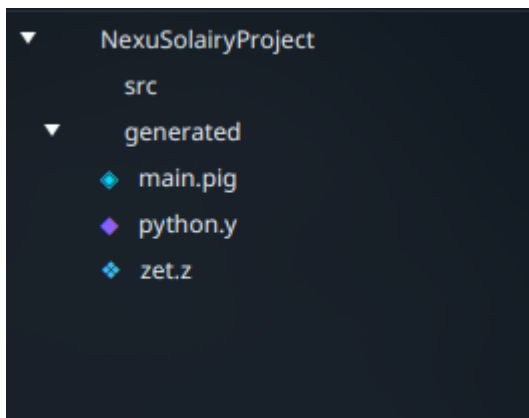


Figura 4: Explorador de proyectos y jerarquía de archivos.

Propósito: Mostrar la estructura jerárquica de carpetas y código fuente del proyecto activo.

Elementos visuales: Árbol de navegación lateral con íconos distintivos según la extensión (.pig, .y, .z).

Interacción: Haga doble clic sobre cualquier archivo para abrirlo en la pestaña correspondiente del editor.

3.3. Barra de Menú Principal y Opciones del IDE

El IDE organiza sus comandos globales en menús desplegable especializados:

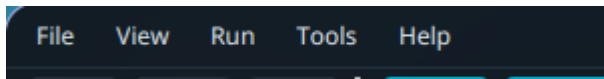


Figura 5: Barra de menú principal.

Propósito: Proporcionar acceso estructurado a todas las funciones globales del sistema.

Elementos visuales: Barra superior con las opciones desplegables File, View, Run, Tools y Help.

Interacción: Haga clic sobre cualquier categoría para desplegar la lista de comandos disponibles.

Menú File

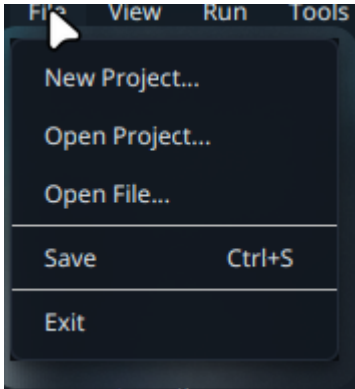


Figura 6: Desplegable del menú File.

Propósito: Gestión de operaciones de archivo e historial.

Elementos visuales: Opciones de New Project, Open Project, Open File, Save y Exit.

Interacción: Seleccione la acción requerida o utilice los atajos de teclado asignados (ej. Ctrl+S).

Menú View

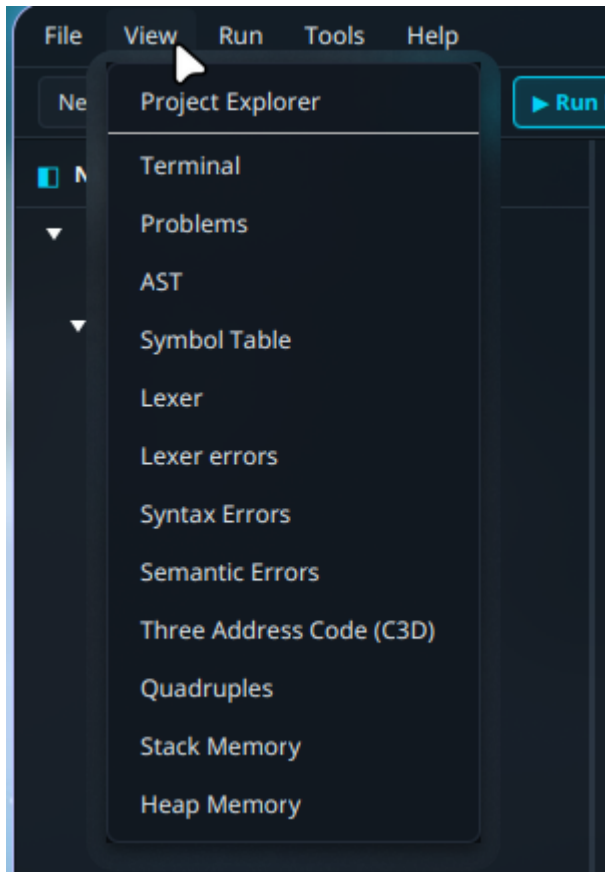


Figura 7: Desplegable del menú View.

Propósito: Control de visibilidad de paneles y herramientas de inspección.

Elementos visuales: Lista de paneles (AST, Symbol Table, Lexer, Syntax, Semantic, C3D, Quadruples, Stack, Heap).

Interacción: Haga clic en un elemento para conmutar o enfocar la vista del panel correspondiente.

Menú Run

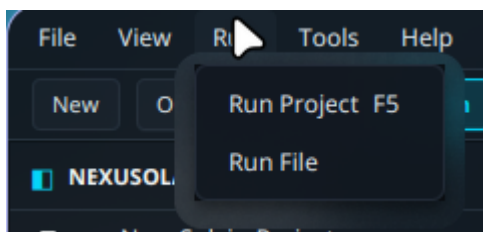


Figura 8: Desplegable del menú Run.

Propósito: Ejecución y procesamiento completo del proyecto o archivo actual.

Elementos visuales: Comandos "Run Project F5" y "Run File".

Interacción: Presione F5 o seleccione "Run Project" para iniciar la compilación y ejecución global.

Menú Tools

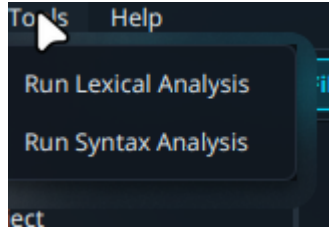


Figura 9: Desplegable del menú Tools.

Propósito: Ejecución aislada de las distintas fases de análisis del compilador.

Elementos visuales: Comandos "Run Lexical Analysis" y "Run Syntax Analysis".

Interacción: Seleccione una fase específica para ejecutar únicamente ese módulo de análisis.

Menú Help

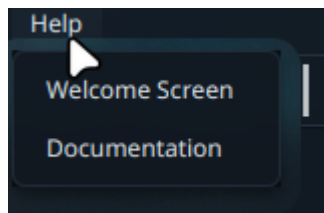


Figura 10: Desplegable del menú Help.

Propósito: Acceso a la documentación interna y reapertura de la pantalla de bienvenida.

Elementos visuales: Opciones "Welcome Screen" y "Documentation".

Interacción: Haga clic en la opción deseada para consultar la ayuda o volver al inicio.

3.4. Barra de Herramientas de Acceso Rápido

Ubicada en la parte superior, facilita el acceso directo a las acciones de desarrollo más frecuentes:

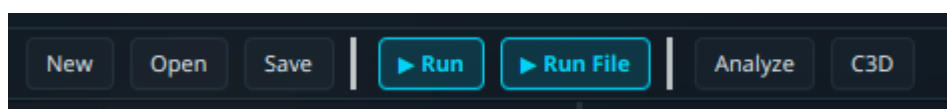


Figura 11: Barra de herramientas de acceso rápido.

Propósito: Proporcionar botones de un solo clic para las operaciones más utilizadas.

Elementos visuales: Botones New, Open, Save, Run, Run File, Analyze y C3D.

Interacción: Haga clic en cualquier botón para disparar la acción inmediata correspondiente.

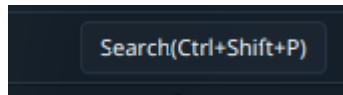
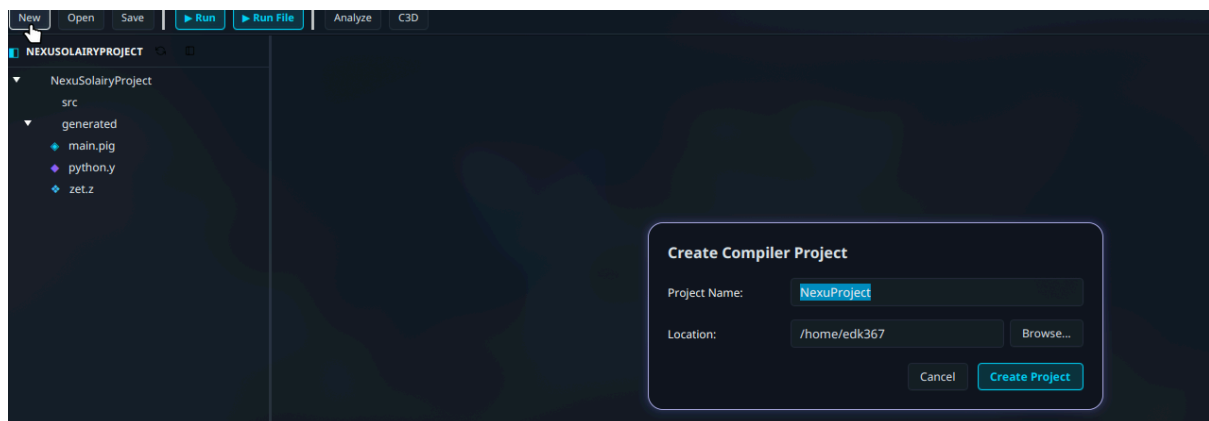


Figura 12: Campo de búsqueda rápida.

Propósito: Permitir la localización rápida de símbolos, comandos y archivos dentro del entorno.

Elementos visuales: Caja de texto de búsqueda con indicación del atajo (Ctrl+Shift+P).

Interacción: Escriba el término a buscar o presione Ctrl+Shift+P para enfocar el campo.



3.5. Editor de Código Multilenguaje

El editor soporta pestañas múltiples para trabajar simultáneamente con diferentes lenguajes.

Cada dialecto cuenta con un esquema de colores e identificadores de estado:

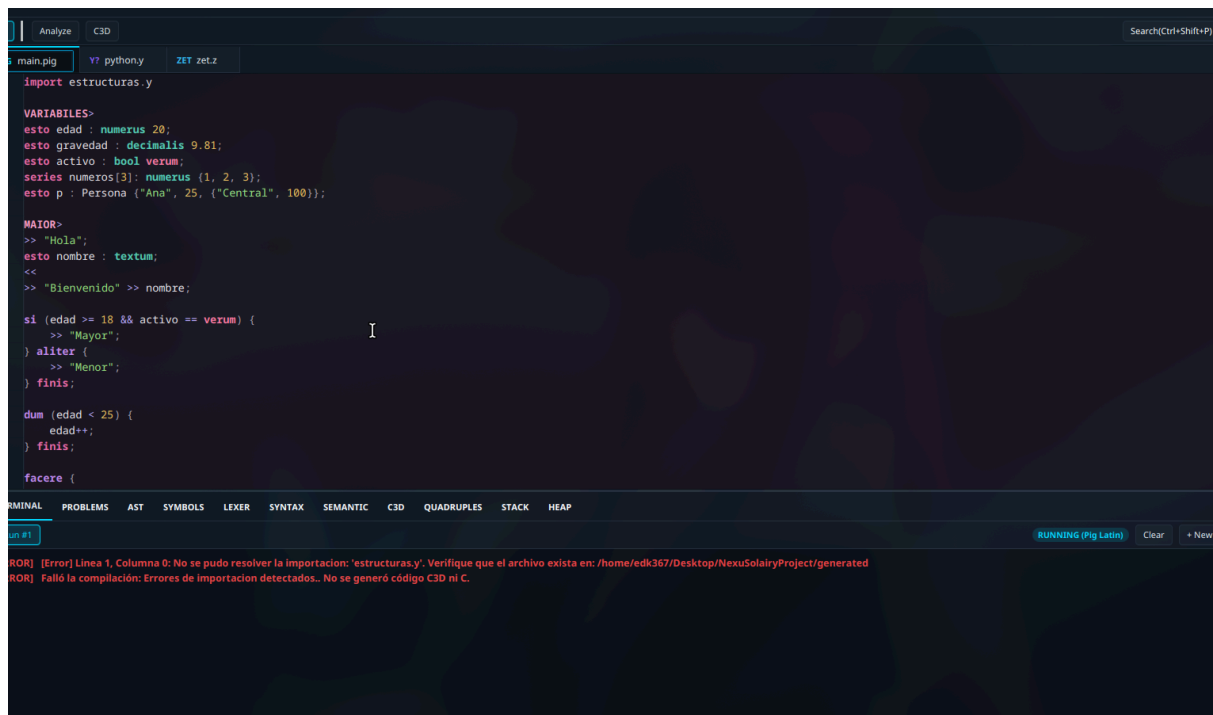


Figura 13: Area principal del editor multilenguaje con código fuente cargado.

Propósito: Proporcionar un espacio de trabajo enriquecido para la edición, formateo y compilación de scripts estelares.

Elementos visuales: Pestañas múltiples de archivos, números de línea, resaltado de sintaxis coloreado y panel inferior de resultados de compilación.

Interacción: Escriba y modifique código fuente directamente; cambie entre archivos mediante las pestañas superiores.

3.6. Panel Inferior de Herramientas y Depuración

El panel inferior integra pestañas especializadas para cada fase de análisis, depuración e inspección de memoria del compilador.

Consola de Terminal y Salida

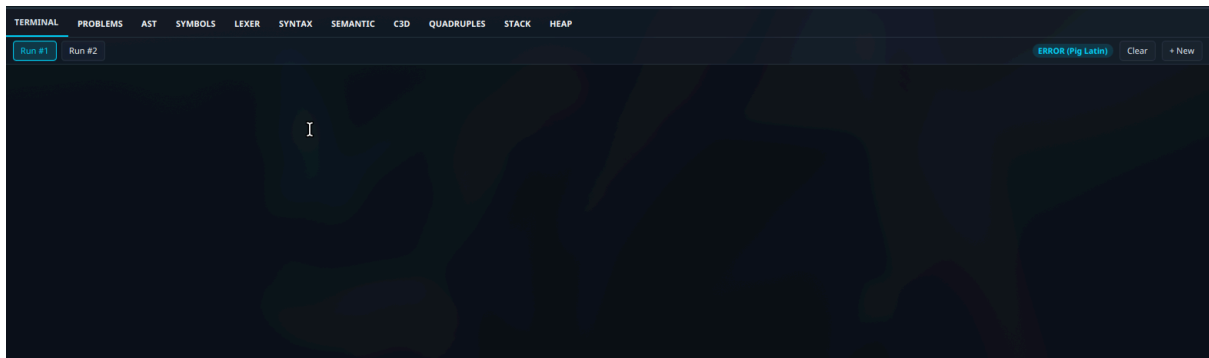


Figura 14: Panel de terminal en estado inicializado.

Propósito: Mostrar el estado del entorno de ejecución virtual de Nexus Solairy.

Elementos visuales: Pestaña Terminal activa con botón de estado "READY" en color cian.

Interacción: Utilice los botones "+ New" para abrir sesiones o "Clear" para limpiar la pantalla.

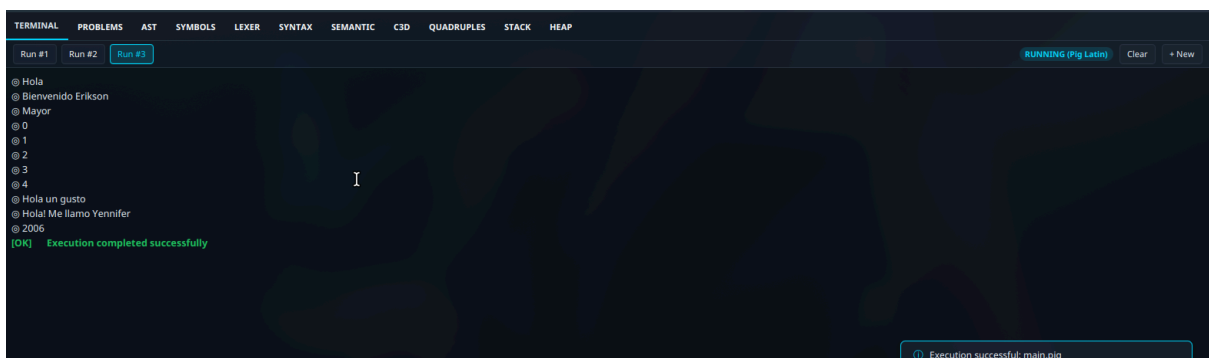


Figura 15: Consola con salida de ejecución completada exitosamente.

Propósito: Desplegar las salidas estándar generadas por la ejecución de los programas compilados.

Elementos visuales: Líneas de texto impresas por el programa y notificación verde "[OK] Execution completed successfully".

Interacción: Examine las impresiones en consola para verificar la lógica de salida del script.

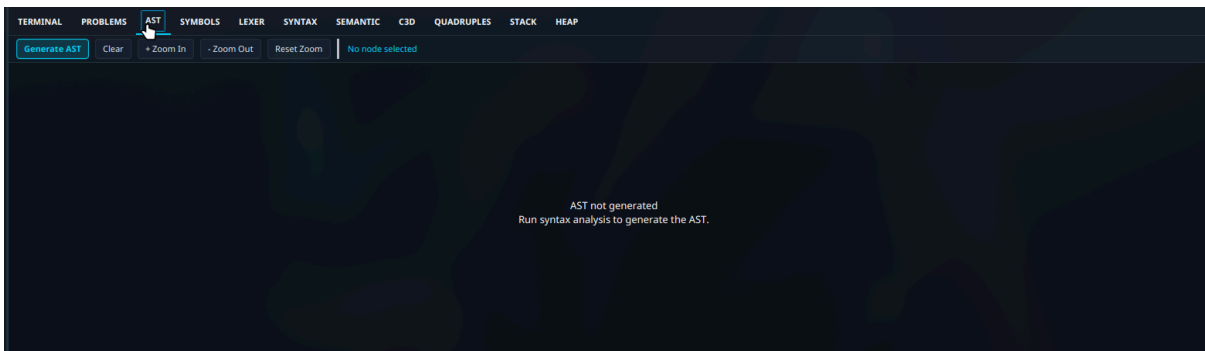
Panel de Problemas y Diagnósticos

Figura 16: Panel de problemas sin errores detectados.

Propósito: Listar advertencias y errores encontrados durante la etapa de compilación.

Elementos visuales: Tabla con columnas Severity, File, Line, Column y Message.

Interacción: Filtre por severidad o haga doble clic sobre un error para navegar a la línea causante.



Visualizador del Árbol Sintáctico (AST)

Figura 17: Panel para la generación del Árbol de Sintaxis Abstracta.

Propósito: Renderizar gráficamente la estructura jerárquica de sintaxis del código analizado.

Elementos visuales: Controles "Generate AST", "+ Zoom In", "- Zoom Out" y lienzo visualizador.

Interacción: Presione "Generate AST" tras el análisis sintáctico y utilice los controles de zoom para navegar la gráfica.

Tabla de Símbolos Registrados

TERMINAL PROBLEMS AST SYMBOLS LEXER SYNTAX SEMANTIC C3D QUADRUPLES STACK HEAP					
Tokens		Lexical Errors			
Filter tokens...		Tokens: 0			
Token		Lexeme		Line	Column
IMPORT		import		1	1
ID		clazz		1	8
DOT		.		1	13
ID		utils		1	14
DOT		.		1	19
ID		AnimalCalculate		1	20
DOT		.		1	35
ID		z		1	36
IMPORT		import		2	1
ID		python		2	8
DOT		.		2	14
ID				2	15

Figura 18: Tabla de símbolos globales y locales.

Propósito: Inspeccionar las identificadores, tipos de datos, ámbitos y valores declarados.

Elementos visuales: Tabla interactiva detallando Name, Data Type, Symbol Kind, Scope, Language, Value, Line y Column.

Interacción: Utilice el campo de filtrado superior para buscar símbolos específicos por nombre o ámbito.

Inspector del Analizador Léxico

Figura 19: Tabla de tokens reconocidos por el lexer.

Propósito: Verificar la separación en componentes léxicos (tokens) realizada por la primera fase de compilación.

Elementos visuales: Columnas con el tipo de Token, Lexeme asociado y ubicación en línea/columna.

Interacción: Explore la lista para validar la correcta categorización de palabras reservadas e identificadores.

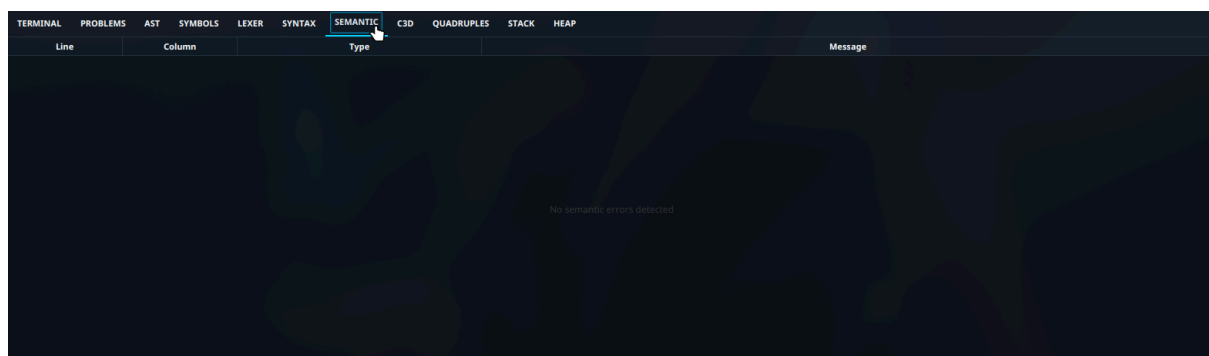
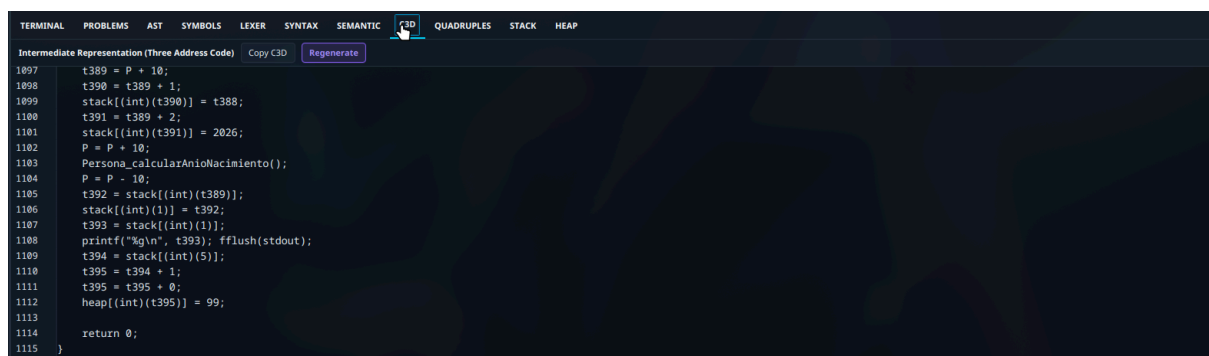


Figura 20: Inspector de errores sintácticos y semánticos.

Propósito: Exhibir la ausencia o detalle de fallos en las reglas de gramática y chequeo de tipos.

Elementos visuales: Mensaje informativo "No syntactic/semantic errors detected" cuando la verificación es exitosa.

Interacción: Cambie entre las pestañas SYNTAX y SEMANTIC para revisar diagnósticos de cada nivel.



Visor de Código de Tres Direcciones (C3D)

Figura 21: Visor del Código de Tres Direcciones generado.

Propósito: Mostrar la representación intermedia en instrucciones simples de tres direcciones.

Elementos visuales: Editor de código de solo lectura numerado con botones "Copy C3D" y "Regenerate".

Interacción: Utilice "Copy C3D" para copiar el texto al portapapeles o "Regenerate" para volver a construir el código intermedio.

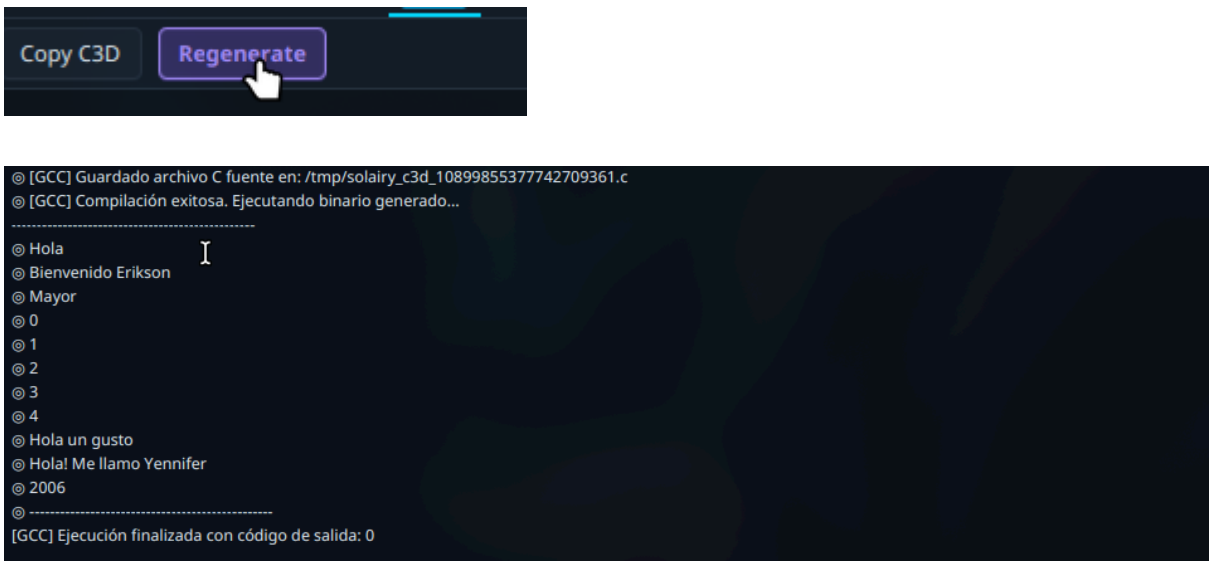


Figura 22: Consola de ejecución compilada con GCC.

Propósito: Presentar el resultado de compilación y ejecución del código C tras procesar el C3D.

Elementos visuales: Logs del compilador C indicando la ruta del archivo temporal y el código de salida.

Interacción: Revise que el código de salida finalice con 0 indicando ejecuciones nativas exitosas.

TERMINAL	PROBLEMS	AST	SYMBOLS	LEXER	SYNTAX	SEMANTIC	C3D	QUADRUPLAS	STACK	HEAP
#		Operator						Arg1	Arg2	Result
0		LABEL								AnimalCalculate_ctor
1		ADD				P		2		t0
2		STACK_READ				t0				t1
3		ADD				P		1		t2
4		STACK_READ				t2				t3
5		ADD				t3		0		t4
6		HEAP_WRITE				t4		t1		
7		ADD				P		3		t5
8		STACK_READ				t5				t6
9		ADD				P		1		t7
10		STACK_READ				t7				t8
11		ADD				t8		1		t9
12		HEAP_WRITE				t9		t6		
13		RETURN								

Tabla de Cuádruplas e Inspectores de Memoria

TERMINAL	PROBLEMS	AST	SYMBOLS	LEXER	SYNTAX	SEMANTIC	C3D	QUADRUPLES	STACK	HEAP
Stack Memory Inspector										
Address		Type		Value		Object Details / References				
stack[0]		entero		heap[0]		Variable global 'edad' [numerus]				
stack[1]		entero		2026		Variable global 'anio' [numerus]				
stack[2]		Puntero a Heap (cadena)		heap[9] -> "in"		Variable global 'gravedad' [decimalis] -> Referencia en heap[9]				
stack[3]		booleano		heap[1]		Variable global 'activo' [boolean]				
stack[4]		booleano		heap[1]		Variable global 'mayor' [boolean]				
stack[5]		Puntero a Heap (numerus[])		heap[0]		Variable global 'numeros' [numerus] -> Base del arreglo en heap[0]				
stack[6]		Puntero a Heap (Usuario)		heap[0]		Variable global 'p' [struct] -> Instancia en heap[0]				
stack[7]		Puntero a Heap (cadena)		heap[11] -> "Erikson"		Variable global 'nombre' [cadena] -> Referencia en heap[11]				
stack[8]		entero		heap[5]		Variable global 'i' [entero]				
stack[9]		Puntero a Heap (Persona)		heap[57]		Variable global 'obj' [class] -> Instancia en heap[57]				
stack[10]		entero		2026		Celda de stack[10]				
stack[11]		Puntero a Heap		heap[57]		Referencia a memoria dinámica en heap[57]				
stack[12]		entero		2026		Celda de stack[12]				

Figura 23: Tabla de representación de Cuádruplas.

Propósito: Estructurar las operaciones en formato de cuatro campos (Operator, Arg1, Arg2, Result).

Elementos visuales: Tabla numerada indicando las operaciones de asignación, saltos y llamadas.

Interacción: Verifique la transformación sintáctica a cuádruplas para propósitos de depuración.

TERMINAL	PROBLEMS	AST	SYMBOLS	LEXER	SYNTAX	SEMANTIC	C3D	QUADRUPLES	STACK	HEAP
Heap Memory Inspector										
Address		Type		Value		Object Details / References				
heap[0]		tamaño arreglo		3 elementos		Arreglo 'numeros' - Capacidad asignada				
heap[1]		elemento arreglo [0]		99		Arreglo 'numeros[0]' (base heap[0])				
heap[2]		elemento arreglo [1]		0		Arreglo 'numeros[1]' (base heap[0])				
heap[3]		elemento arreglo [2]		0		Arreglo 'numeros[2]' (base heap[0])				
heap[4]		caracter (ASCII)		'H' (72)		Cadena "Hola" [índice 0, base heap[4]]				
heap[5]		caracter (ASCII)		'o' (111)		Cadena "Hola" [índice 1, base heap[4]]				
heap[6]		caracter (ASCII)		'l' (108)		Cadena "Hola" [índice 2, base heap[4]]				
heap[7]		caracter (ASCII)		'a' (97)		Cadena "Hola" [índice 3, base heap[4]]				
heap[8]		fin de cadena (-1)		-1 (EOF/NULL)		Terminador de cadena "Hola" (base heap[4])				
heap[9]		caracter (ASCII)		'n' (10)		Cadena "n" [índice 0, base heap[9]]				
heap[10]		fin de cadena (-1)		-1 (EOF/NULL)		Terminador de cadena "n" (base heap[9])				
heap[11]		caracter (ASCII)		'E' (69)		Cadena "Erikson" [índice 0, base heap[11]]				
heap[12]		caracter (ASCII)		'r' (114)		Cadena "Erikson" [índice 1, base heap[11]]				

Figura 24: Inspector de memoria Stack.

Propósito: Visualizar el estado de la pila de memoria durante la simulación de ejecución.

Elementos visuales: Filas organizadas por Address (stack[n]), Type, Value y Object Details / References.

Interacción: Inspeccione las posiciones de memoria para rastrear variables locales y punteros.

Figura 25: Inspector de memoria Heap.

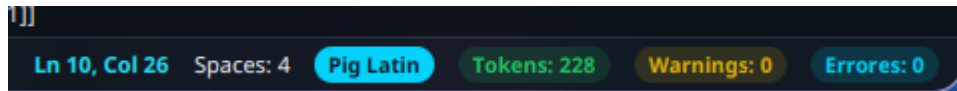
Propósito: Monitorear la asignación dinámica de objetos, arreglos y cadenas de texto.

Elementos visuales: Tabla detallando direcciones heap[n], tipos de elementos almacenados, valores ASCII/numéricos y botón "Trigger GC".

Interacción: Presione "Trigger GC" para simular la recolección de basura o examine el contenido de los arreglos almacenados.

Barra de Estado Inferior

Figura 26: Barra de estado e indicadores del editor.



Propósito: Brindar métricas en tiempo real sobre la posición del cursor y el conteo de tokens/errores del archivo activo.

Elementos visuales: Etiquetas con número de línea/columna, dialecto detectado (ej. Pig Latin), total de tokens, advertencias y errores.

Interacción: Consulte este panel para verificar de un vistazo la salud sintáctica del documento en edición.